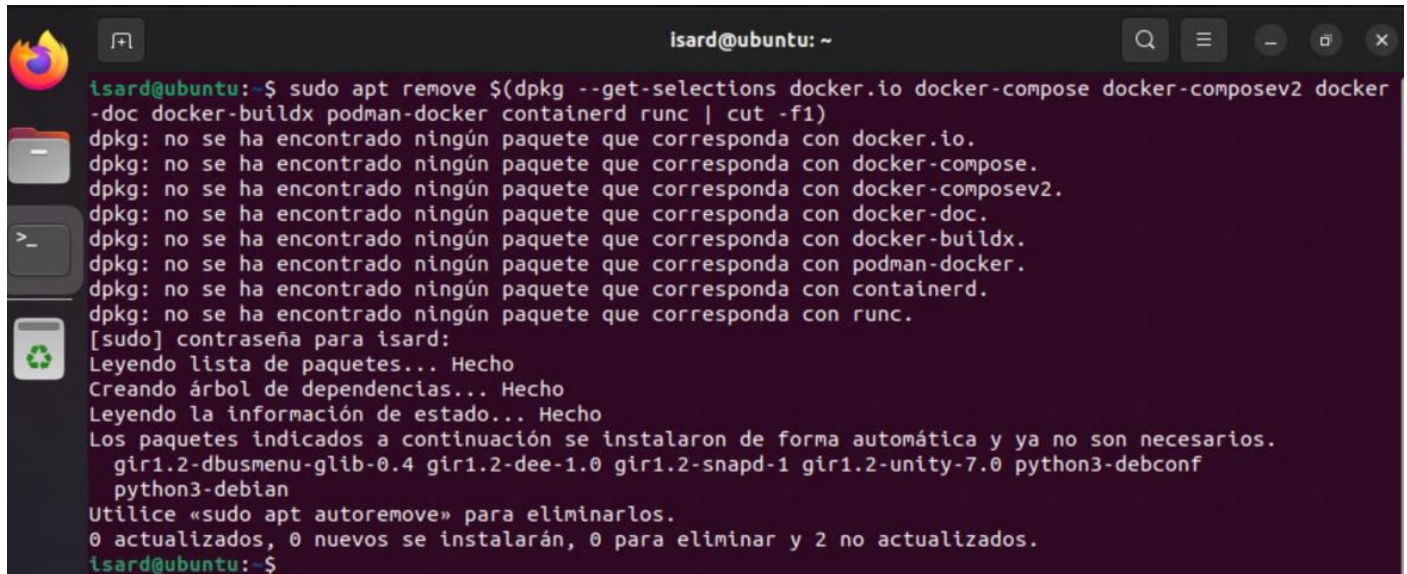


Instalación de Docker en Linux

Antes de comenzar, es necesario eliminar las versiones antiguas y paquetes para que no generen conflictos, para eliminarlos, se debe ejecutar:

```
sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-compose-v2  
docker-doc docker-buildx podman-docker containerd runc | cut -f1)
```

A terminal window titled 'isard@ubuntu: ~' showing the execution of a command to remove old Docker packages. The command is: `sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-compose-v2 docker-doc docker-buildx podman-docker containerd runc | cut -f1)`. The output shows that no packages were found for removal. The terminal also shows the standard Ubuntu update process: 'Leyendo lista de paquetes... Hecho', 'Creando árbol de dependencias... Hecho', 'Leyendo la información de estado... Hecho'. It then lists several packages that were installed automatically and are no longer needed: `gir1.2-dbusmenu-glib-0.4 gir1.2-dee-1.0 gir1.2-snapd-1 gir1.2-unity-7.0 python3-debconf python3-debian`. It suggests using `sudo apt autoremove` to remove them. The final status is: '0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.' The prompt returns to `isard@ubuntu:~$`.

```
isard@ubuntu:~$ sudo apt remove $(dpkg --get-selections docker.io docker-compose docker-compose-v2 docker-doc docker-buildx podman-docker containerd runc | cut -f1)
dpkg: no se ha encontrado ningún paquete que corresponda con docker.io.
dpkg: no se ha encontrado ningún paquete que corresponda con docker-compose.
dpkg: no se ha encontrado ningún paquete que corresponda con docker-composev2.
dpkg: no se ha encontrado ningún paquete que corresponda con docker-doc.
dpkg: no se ha encontrado ningún paquete que corresponda con docker-buildx.
dpkg: no se ha encontrado ningún paquete que corresponda con podman-docker.
dpkg: no se ha encontrado ningún paquete que corresponda con containerd.
dpkg: no se ha encontrado ningún paquete que corresponda con runc.
[sudo] contraseña para isard:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  gir1.2-dbusmenu-glib-0.4 gir1.2-dee-1.0 gir1.2-snapd-1 gir1.2-unity-7.0 python3-debconf
  python3-debian
Utilice «sudo apt autoremove» para eliminarlos.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 2 no actualizados.
isard@ubuntu:~$
```

Antes de instalar docker, se configura **apt** el repositorio de docker con los siguientes comandos:

Primero se prepara el sistema y se descarga la clave de seguridad oficial de docker, esto sirve para garantizar la seguridad indicando que Ubuntu confíe en las actualizaciones de docker.

Sudo apt update

Instalar la herramienta necesaria para para verificar certificados de seguridad SSL.

Sudo apt install ca-certificates curl

Se crea una carpeta segura para las llaves.

```
sudo install -m 0755 -d /etc/apt/keyrings
```

Se descarga la clave criptográfica de Docker.

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o  
/etc/apt/keyrings/docker.asc
```

Se dan los permisos de lectura a la clave.

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```

Descargados 627 kB en 0s (4.315 kB/s)
Preconfigurando paquetes ...
(Leyendo la base de datos ... 185792 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../ca-certificates_20260601~22.04.1_all.deb ...
Desempaquetando ca-certificates (20260601~22.04.1) sobre (20230311ubuntu0.22.04.1) ...
Preparando para desempaquetar .../curl_7.81.0-1ubuntu1.29_amd64.deb ...
Desempaquetando curl (7.81.0-1ubuntu1.29) sobre (7.81.0-1ubuntu1.16) ...
Preparando para desempaquetar .../libcurl4_7.81.0-1ubuntu1.29_amd64.deb ...
Desempaquetando libcurl4:amd64 (7.81.0-1ubuntu1.29) sobre (7.81.0-1ubuntu1.16) ...
Configurando ca-certificates (20260601~22.04.1) ...
Updating certificates in /etc/ssl/certs...
rehash: warning: skipping ca-certificates.crt,it does not contain exactly one certificate or CRL
24 added, 40 removed; done.
Configurando libcurl4:amd64 (7.81.0-1ubuntu1.29) ...
Configurando curl (7.81.0-1ubuntu1.29) ...
Procesando disparadores para man-db (2.10.2-1) ...
Procesando disparadores para libc-bin (2.35-0ubuntu3.7) ...
Procesando disparadores para ca-certificates (20260601~22.04.1) ...
Updating certificates in /etc/ssl/certs...
0 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
isard@ubuntu:~$ sudo install -m 0755 -d /etc/apt/keyrings
isard@ubuntu:~$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker
.asc
isard@ubuntu:~$ sudo chmod a+r /etc/apt/keyrings/docker.asc
isard@ubuntu:~$

```

Luego en el archivo `docker.sources`, se agrega el repositorio oficial de docker a la configuracion de Ubuntu.

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
```

Types: deb

URIs: <https://download.docker.com/linux/ubuntu>

Suites: `$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")`

Components: stable

Architectures: `$(dpkg --print-architecture)`

Signed-By: `/etc/apt/keyrings/docker.asc`

EOF

Por último, se ejecuta de nuevo **sudo apt update**.

```

isard@ubuntu:~$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: jammy
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
isard@ubuntu:~$

```

Se instalan los paquetes de Docker.

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

```
isard@ubuntu:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compo
se-plugin
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
  gir1.2-dbusmenu-glib-0.4 gir1.2-dee-1.0 gir1.2-snapd-1 gir1.2-unity-7.0 python3-debconf
  python3-debian
Utilice «sudo apt autoremove» para eliminarlos.
Se instalarán los siguientes paquetes adicionales:
  docker-ce-rootless-extras pigz
Paquetes sugeridos:
  cgroupfs-mount | cgroup-lite docker-model-plugin
Se instalarán los siguientes paquetes NUEVOS:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras
  docker-compose-plugin pigz
0 actualizados, 7 nuevos se instalarán, 0 para eliminar y 508 no actualizados.
Se necesita descargar 101 MB de archivos.
Se utilizarán 383 MB de espacio de disco adicional después de esta operación.
```

- **¿Cuál es la diferencia entre instalar Docker Engine y Docker Desktop?**

Docker Engine es el motor para que funciones Docker mientras que Docker Desktop incluye Docker Engine más una interfaz gráfica y herramientas adicionales. Para instalar Docker Engine, se hace mediante comandos, a diferencia de Docker Desktop se instala mediante un asistente gráfico automatizado.

- **Docker solo funciona en Linux, ¿podemos trabajar en Windows? ¿Cómo?**

Se puede trabajar con Windows mediante WSL (Windows Subsystem for Linux) este crea una máquina virtual ligera con Linux, dentro de ella funciona Docker.

- **¿Cómo puedes visualizar el contenido de docker.sources? Muéstralo.**

Anteriormente se usaban los archivos .list para repositorios, ahora se adoptó nuevo estándar usando archivos .sources

cat /etc/apt/sources.list.d/docker.sources

```
isard@ubuntu:/etc/apt/sources.list.d$ cat /etc/apt/sources.list.d/docker.sources
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: jammy
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
isard@ubuntu:/etc/apt/sources.list.d$
```

- Ejecuta en tu máquina con la instalación completa de Docker el comando *docker run hello-world* y haz una captura de pantalla con el resultado de la ejecución del comando.

```
ades Terminal 30 de sep 18:53
isard@ubuntu: /etc/apt/sources.list.d
isard@ubuntu:/etc/apt/sources.list.d$ docker run hello-world
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
isard@ubuntu:/etc/apt/sources.list.d$
```

- ¿Puedes ejecutarlo como usuario sin privilegios? ¿Por qué? ¿Qué se te ocurre hacer para no tener que trabajar con el prompt del root?

Al tratar de ejecutar **docker run hello-world** da error de privilegios, esto es así porque por defecto el demonio del motor se ejecuta con privilegios de root, para solucionarlo, se debe añadir el usuario al grupo docker. Para ello se ejecutan los siguientes comandos:

sudo groupadd Docker

sudo usermod -aG docker \$USER

newgrp docker

```
ades Terminal 30 de sep 19:00
isard@ubuntu: /etc/apt/sources.list.d
isard@ubuntu:/etc/apt/sources.list.d$ sudo groupadd Docker
isard@ubuntu:/etc/apt/sources.list.d$ sudo usermod -aG docker $USER
isard@ubuntu:/etc/apt/sources.list.d$ newgrp docker
isard@ubuntu:/etc/apt/sources.list.d$
```

- Explica con tus palabras qué ocurre cuando se ejecuta correctamente **docker run hello-world**

Cuando se ejecuta por primera vez, primero busca la imagen hello-world, al no tenerla la descarga y luego ejecuta la imagen.



Isard@ubuntu: ~

```
Isard@ubuntu:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

- Vuelve a ejecutar `docker run hello-world`, ¿hay alguna diferencia con la ejecución anterior? ¿Qué ha ocurrido?

Al ejecutar el comando por 2 vez, no descarga la imagen porque ya la tiene.

```
isard@ubuntu:~$ docker run hello-world
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

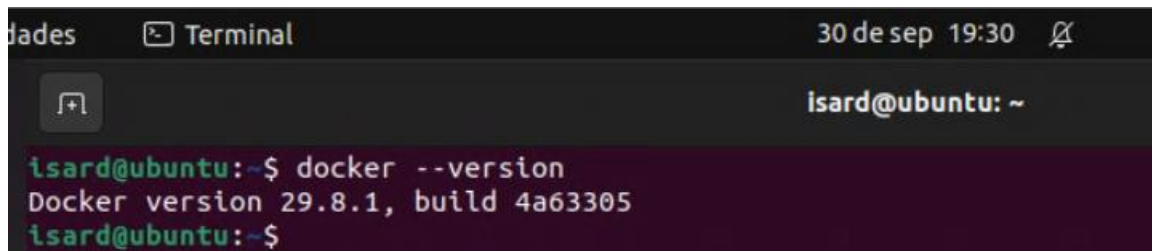
For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

- Ejecuta los siguientes comandos e interpreta su salida:

- **docker --version**

Lo que hace el comando es mostrar solo la versión de Docker del cliente y el número de compilación.

A screenshot of a terminal window titled "Terminal" with a timestamp of "30 de sep 19:30". The prompt is "isard@ubuntu: ~". The command entered is "docker --version" and the output is "Docker version 29.8.1, build 4a63305".

```
isard@ubuntu:~$ docker --version
Docker version 29.8.1, build 4a63305
isard@ubuntu:~$
```

- **docker versión**

Lo que hace el comando es mostrar información más completa que la anterior, muestra un reporte detallado dividido en 2 secciones: cliente y el servidor/motor

```
ades Terminal 30 de sep 19:29 isard@ubuntu: ~
isard@ubuntu:~$ docker version
Client: Docker Engine - Community
 Version:      29.8.1
 API version:  1.56
 Go version:   go1.26.8
 Git commit:   4a63305
 Built:        Tue Sep 15 16:25:26 2026
 OS/Arch:      linux/amd64
 Context:      default

Server: Docker Engine - Community
 Engine:
  Version:      29.8.1
  API version:  1.56 (minimum version 1.40)
  Go version:   go1.26.8
  Git commit:   464cd50
  Built:        Tue Sep 15 16:25:26 2026
  OS/Arch:      linux/amd64
  Experimental: false
 containerd:
  Version:      v2.3.6
  GitCommit:    ee2735368117d2eb259779949d5e75cdafec9761
 runc:
  Version:      1.5.1
  GitCommit:    v1.5.1-0-g8f2685a4
 docker-init:
  Version:      0.19.0
  GitCommit:    de40ad0
```

- **docker info**

El comando muestra el estado de salud actual, estadísticas de contenedores, configuración interna, etc.

```
ades 30 de sep 19:34 isard@ubuntu: ~
isard@ubuntu:~$ docker info
Client: Docker Engine - Community
Version: 29.8.1
Context: default
Debug Mode: false
Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version: v0.37.1
    Path: /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version: v5.5.1
    Path: /usr/libexec/docker/cli-plugins/docker-compose

Server:
Containers: 2
  Running: 0
  Paused: 0
  Stopped: 2
Images: 1
Server Version: 29.8.1
Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: systemd
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
  /etc/cdi
  /var/run/cdi
Swarm: inactive
Runtimes: io.containerd.runc.v2 runc
Default Runtime: runc
Init Binary: docker-init
containerd version: ee2735368117d2eb259779949d5e75cdafec9761
runc version: v1.5.1-0-g8f2685a4
init version: 18.0.0
```